Tree based methods

Lecture 11

Course of:

Statistics and data analysis

16/04/2026

Federico De Guio - Martino Borsato

Trees for regression and classification

- James, cap.8

Trees basics

- Decision Trees are a popular **supervised machine learning technique**, commonly used in many fields of data analysis, including high energy physics
  - First developed and formalised by Breiman et al. 90s
  - Proposed the CART algorithm (Classification And Regression Trees)
  - Became very popular in HEP after use in MiniBooNE experiment (arXiv: 0408124)

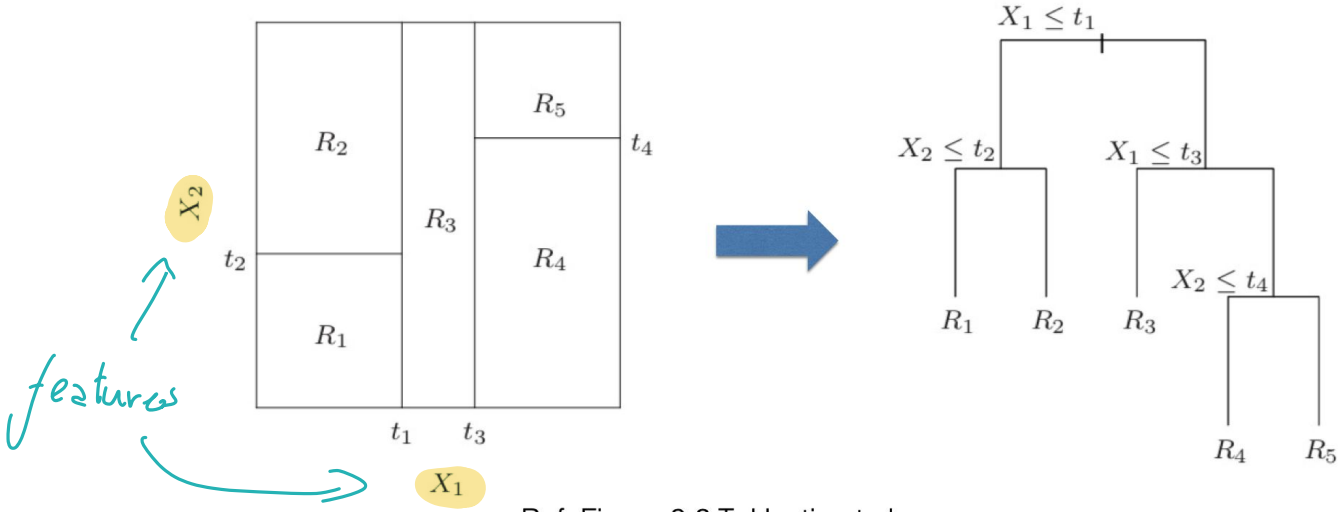
select a region of hyperspace where param checks all conditions

- **Basic concept**: extend the cut-based analysis by not rejecting events that fail a particular criterion, instead, check if other criteria may help to classify these events correctly

- Decision trees can be applied to both **classification** and **regression** problems
- The basic idea is to **chain a set of splitting rules** in order to segment the predictor space
  - High interpretability

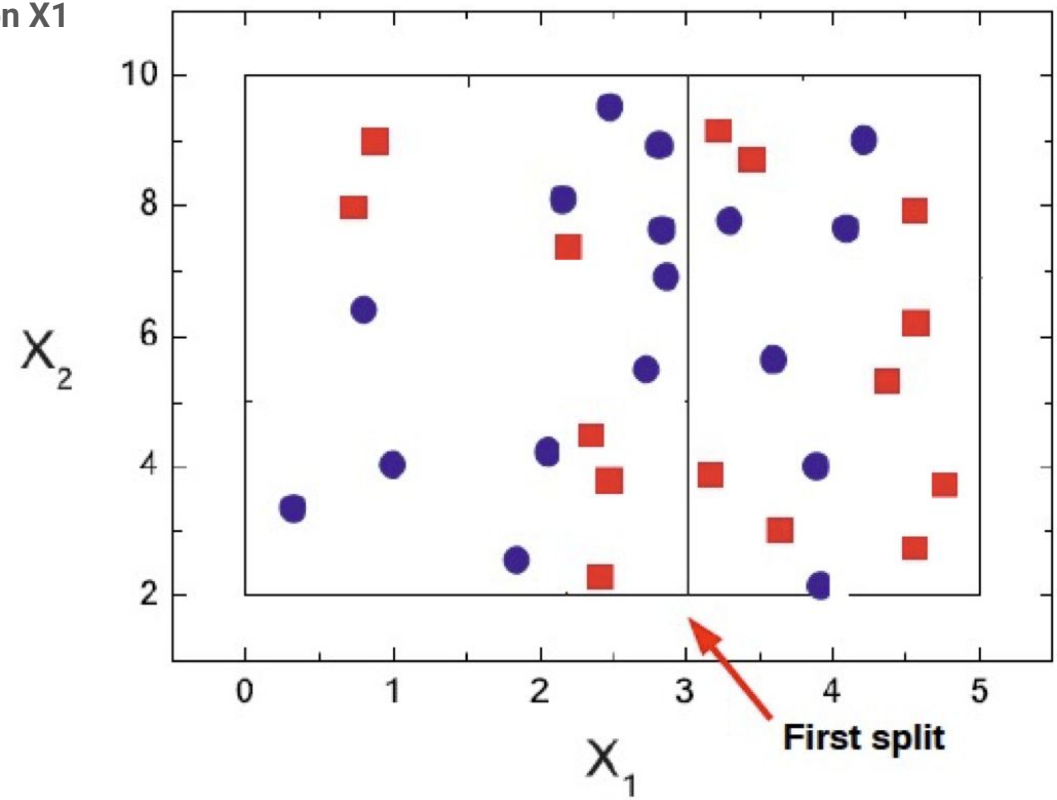
Binary tree

- Perform **recursive binary partitions of the feature space** into a set of rectangles

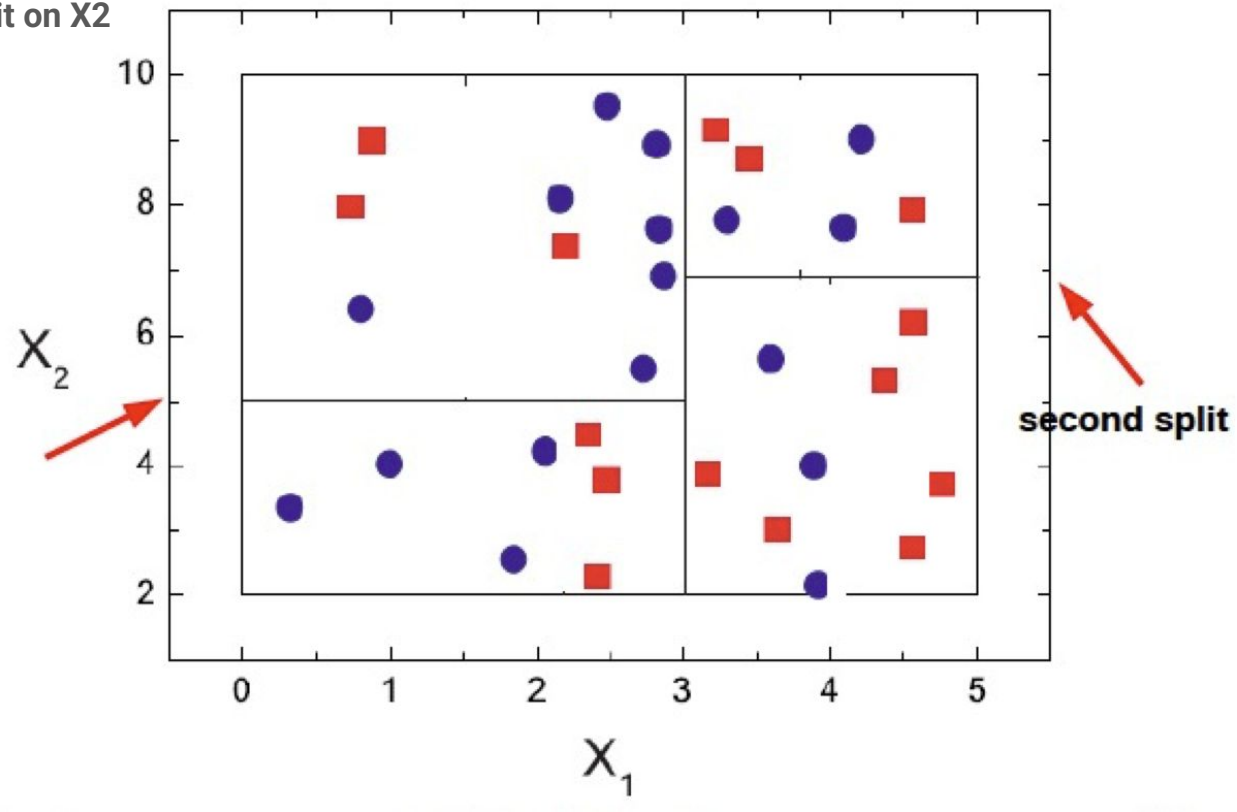


Ref: Figure 9.2,T. Hastie et al.

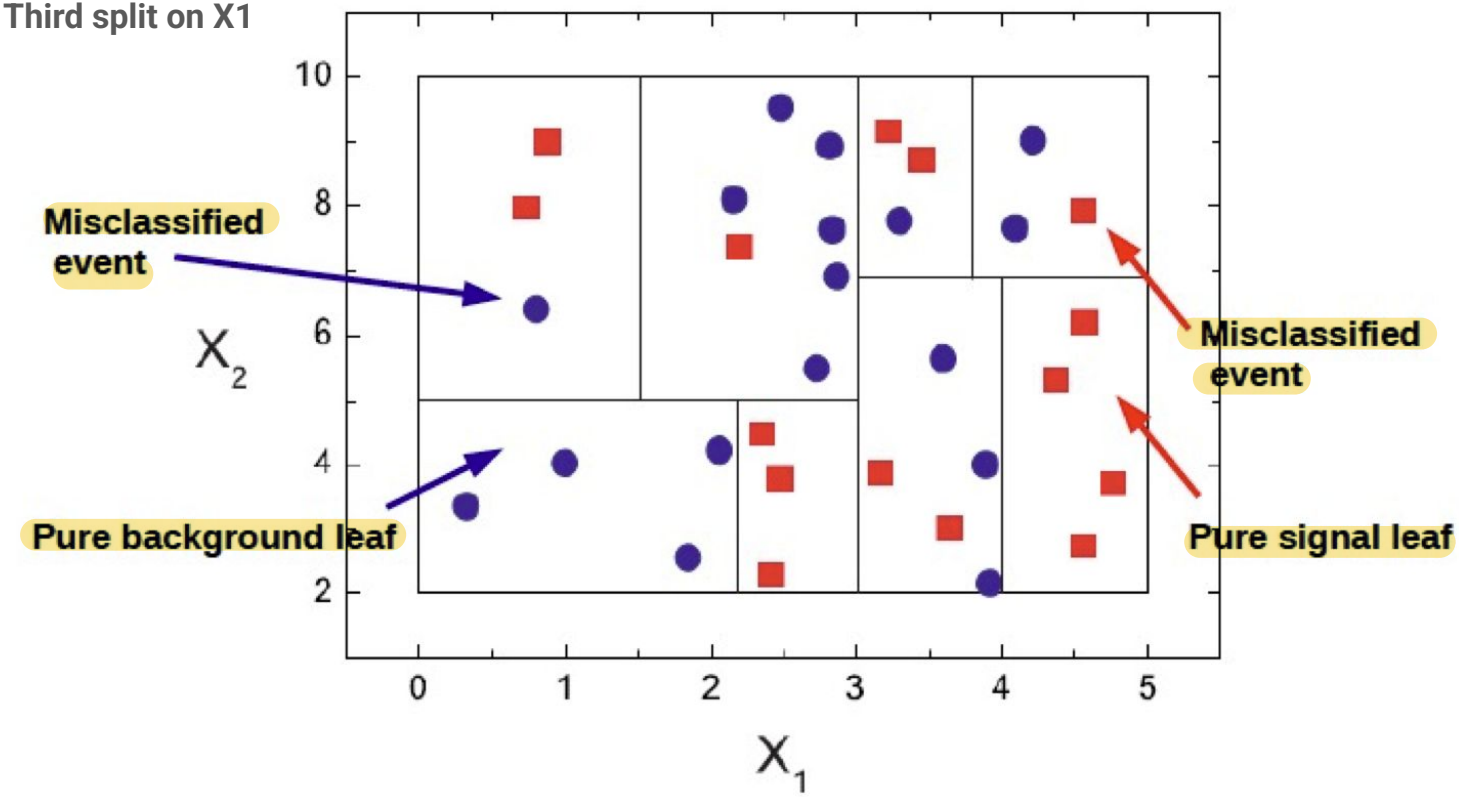
First split on X1



Second split on X2

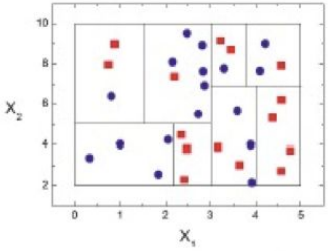


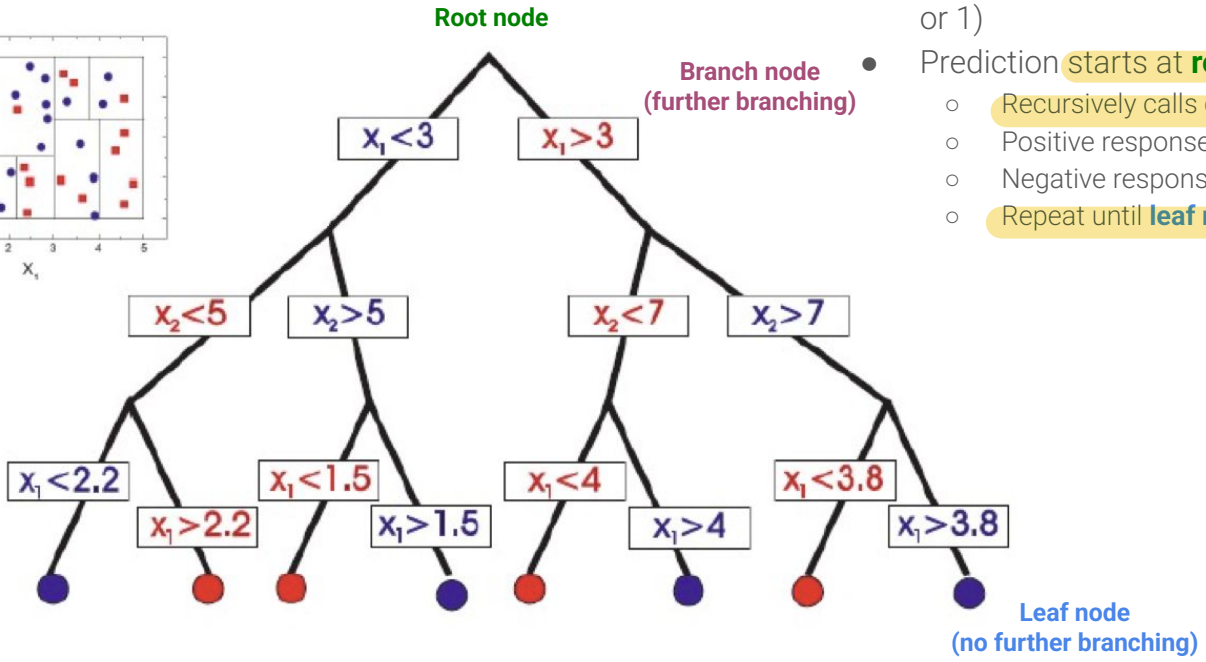
Third split on X1



Tree diagram for binary split

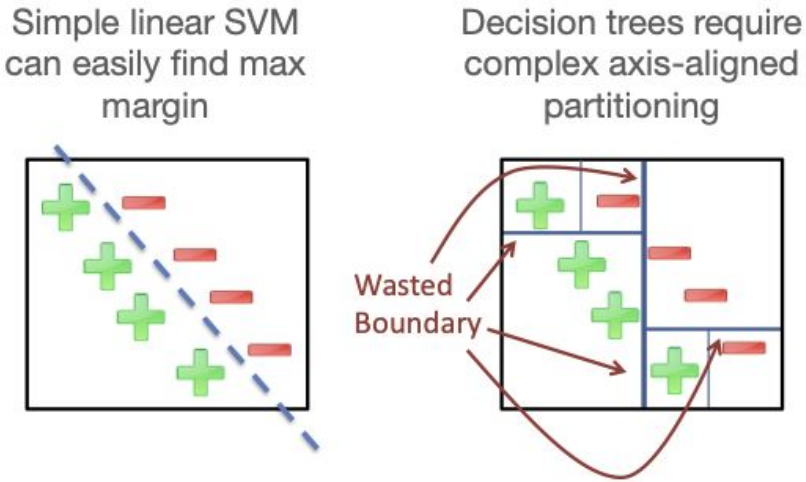
- Every **internal/branch node** has a binary query function **q(x)**
- Every leaf node has a prediction (0 or 1)
- Prediction starts at **root node**
  - Recursively calls **query function**
  - Positive response → left child
  - Negative response → right child
  - Repeat until **leaf node**





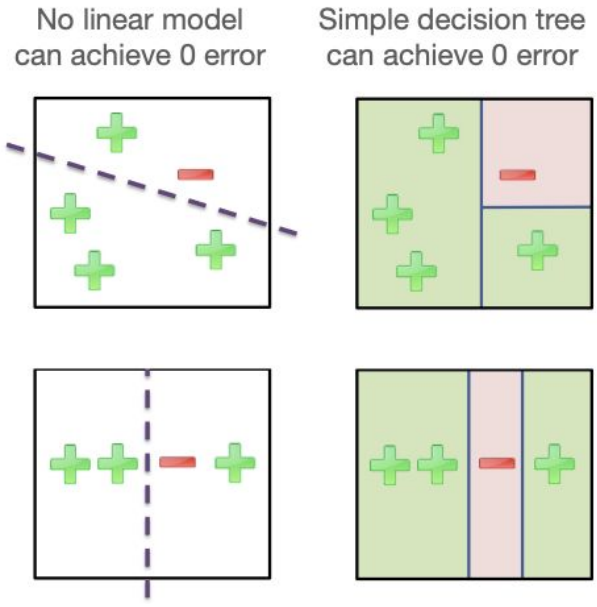
Decision trees vs linear models

- Decision trees are **axis-aligned!**
  - No diagonals



Decision trees are **non-linear models!**

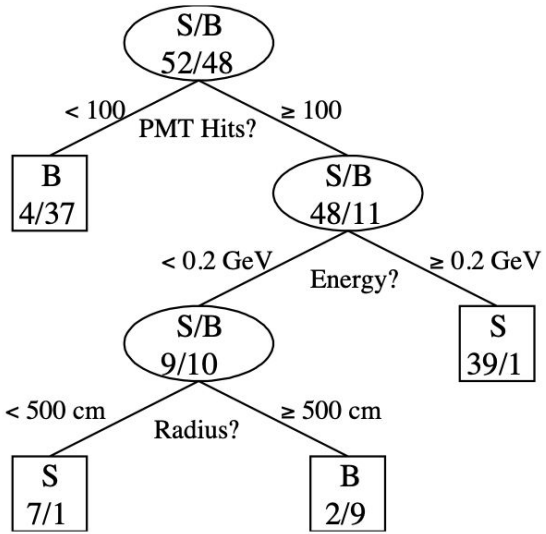
- Decision Trees are **often more accurate!**
- Non-linearity is typically more important
  - Just use many axis-aligned boundaries to approximate diagonal boundaries
  - It's OK to waste model capacity
- **Catch**: requires sufficient training data

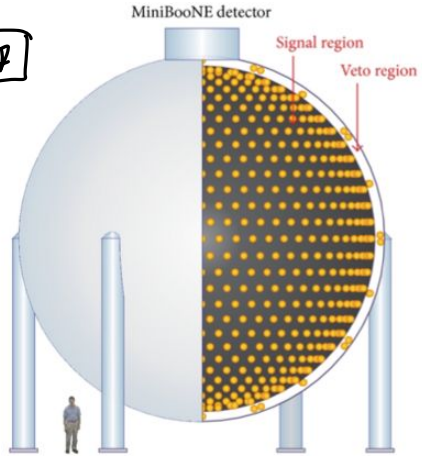


An example from HEP - MiniBooNE experiment

2004

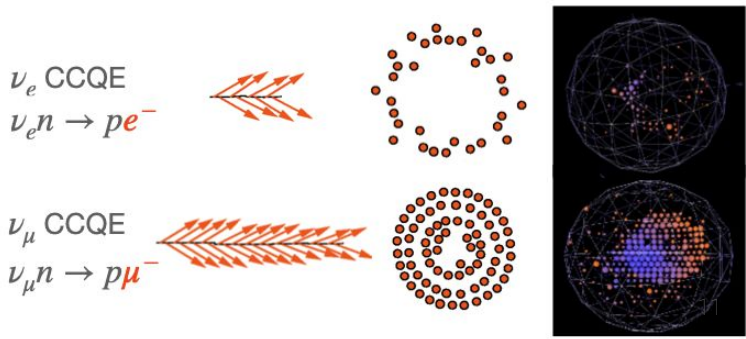
- Leaf nodes classify events as either signal (ν_e) or background (ν_μ)
  - arXiv:physics/0408124





MiniBooNE: 1520 photomultiplier signals

Goal: separate ν_e and ν_μ events



Question:

What tree configuration is expected to give

low branches

new dataset

High bias?

High variance?

high branches

new dataset

⇓

big errors

Growing a decision tree

- How many possible queries?
- How do we choose the best query?

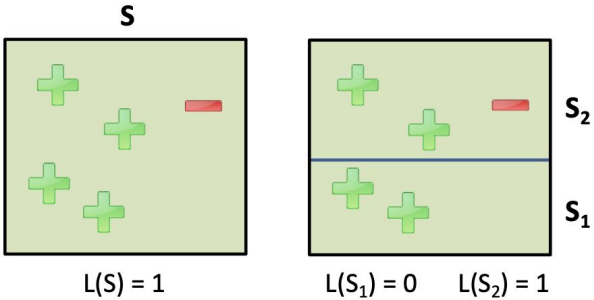
- Define an **impurity function** → the easiest approach would be a **0/1 loss:**

$$L(S') = \min_{f(x)\in\{0,1\}} \sum_{(x,y)\in S} \delta[f(x) \neq y]$$

1 of the two

not [0, 1]

Classification error of best single prediction in each partition



Impurity Reduction = 0

No Benefit From This Split!

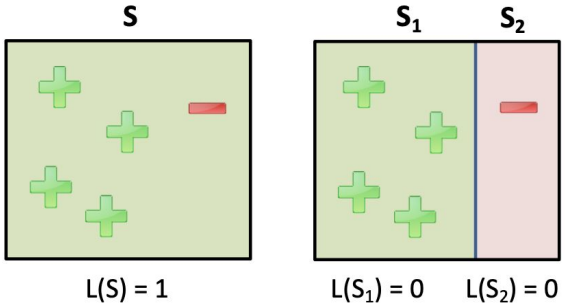
Growing a decision tree

- How many possible queries?
- How do we choose the best query?

- Define an **impurity function** → the easiest approach would be a **0/1 loss:**

$$L(S') = \min_{f(x)\in\{0,1\}} \sum_{(x,y)\in S} \delta[f(x) \neq y]$$

Classification error of best single prediction in each partition



Impurity Reduction = 1

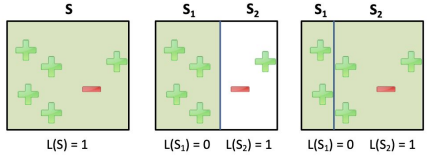
Choose Split with largest impurity reduction!

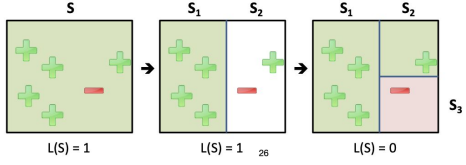
Impurity as a loss function

- Training goal: find decision tree with **low impurity**
- Impurity over leaf nodes = training loss

$$L(S) = \sum_{S' \subset S} L(S') \text{ where } L(S') = \min_{f(x) \in \{0,1\}} \sum_{(x,y) \in S} \delta[f(x) \neq y]$$

- However:
  - 0/1 loss is **discontinuous**
  - **Degeneracies**: all partitioning give same impurity reduction
  - A good partitioning choice may not improve 0/1 loss, e.g., leads to an accurate model with a **subsequent split**





Continuous impurity measures

- **Entropy** / information gain:

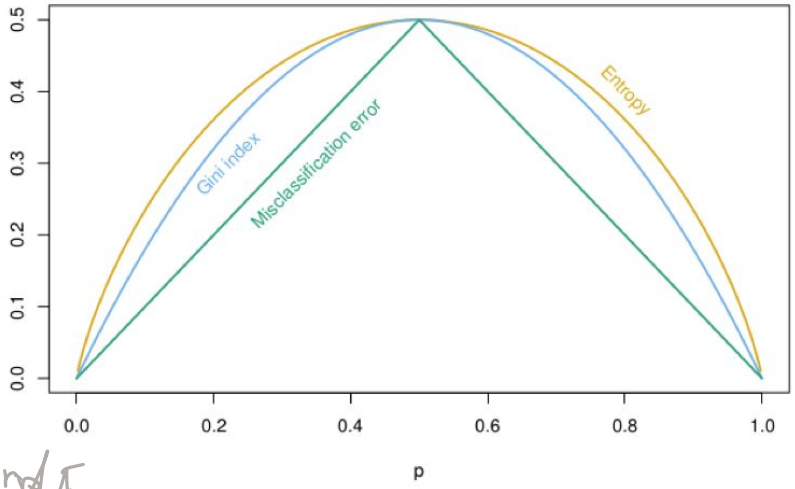
$$L(S') = -|S'|\left(p_{S'}\log p_{S'} + (1-p_{S'})\log(1-p_{S'})\right)$$

- **Gini index** / impurity:

$$L(S') = |S'|\left(1 - p_{S'}^2 - (1-p_{S'})^2\right)$$

probability of getting class right

sample size



The **greedy** approach

- The criterion used to grow a tree chooses the **'optimal' predictor on which to split and chooses the 'optimal' threshold value for splitting** → doesn't care about the global picture

- At each branch node:
  - Scan all features
  - Scan all values for each feature
  - Compute the loss (impurity)
  - Select the cut that minimizes the loss

- **No lookahead** → misses the global optimum
  - However a global optimization would require to generate all the possible combinations and would be unfeasible!

Quantum computing

When/how to **stop splitting**?

- When do we stop growing a tree?
  - When all the nodes are pure? **No, that's overfitting!**
  - Stopping conditions needed (**regularizers**)

- Some option for **stopping conditions** include:
  - **Minimum size**: do not split if resulting children are smaller than a minimum size
  - **Maximum depth**: do not split if the resulting children are beyond some maximum tree depth
  - **Maximum number of nodes**: do not split if tree already has maximum number of allowable nodes
  - **Minimum reduction in impurity**: do not split if resulting children do not reduce impurity by at least δ %

Alternatives to stopping conditions: **cross validation**

- Pre-defined stopping conditions may be suboptimal and **stop too early or too late**..

- → can use **cross validation to choose what combination of stopping conditions is the best** on the validation dataset
  - Define a grid of stopping conditions
  - For each point on the grid, run cross validation and measure the performance on the validation dataset
  - Compare the performance from all the params combinations on the grid
  - Pick the best combination

like depth, leaf, purity

[...] [ ] [ ]

check

Alternatives to stopping conditions: **pruning**

- Rather than preventing a complex tree from growing, we can **obtain a simpler tree by 'pruning' a complex one**

- Standard approach: **cost complexity pruning:**

$$R_\alpha(T) = \text{Error}(T) + \alpha|T|$$

Tree total error
→ larger the tree, smaller the error

Total number of branch nodes
→ complexity of the tree

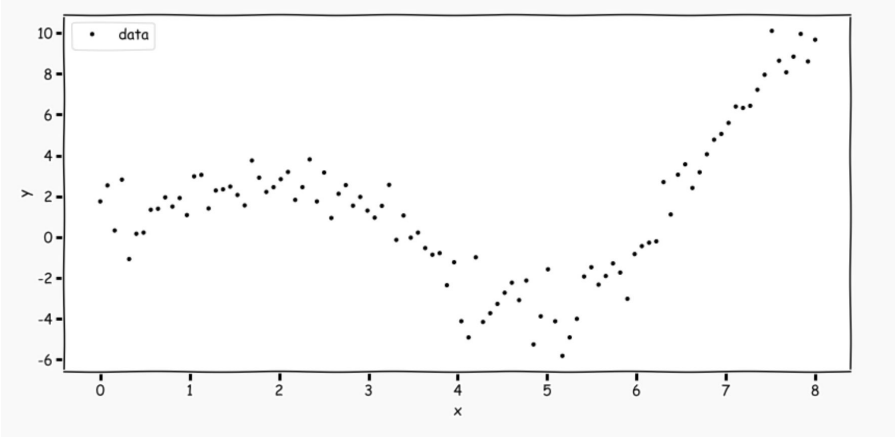
Complexity parameter
→ tax on additional nodes

- **Test several values for alpha with cross validation** → find the best compromise between generalization and accuracy

Do trees work for **regression** as well?

- How can this decision tree approach apply to a regression problem?
  - A **quantitative outcome** is expected

- Questions to consider:
  - What would be a reasonable **loss function**?
  - How would you determine any **splitting criteria**?
  - How would you perform prediction in each leaf?



Adaptation for regression → 2 modifications needed

1. The splitting criteria we've examined promoted splits that were pure → new regions increasingly specialized in a single class
   - **For classification**, purity of the regions is a good indicator the performance of the model
   - **For regression**, we want to select a splitting criterion that promotes splits that improves the predictive accuracy of the model as measured by, say, the **MSE**

2. For regression with output in $\mathbb{R}$, we want to label each region in the model with a **real number** - typically the average of the output values of the training points contained in the region

Training regression trees

- Start with an empty decision tree (undivided features space)
- Consider **each feature** and **each value** for a possible cut
- The best condition is selected by **minimizing the total loss** of the split
  - The weighted average of the MSE left and right is computed

MSE is a common choice

$$MSE(S') = \frac{1}{|S'|}\sum_{i \in S'}(y_i - \bar{y}_{S'})^2$$

$$Loss_{split} = \frac{|S_L|}{|S_{father}|}\mathrm{MSE}(S_L) + \frac{|S_R|}{|S_{father}|}MSE(S_R)$$

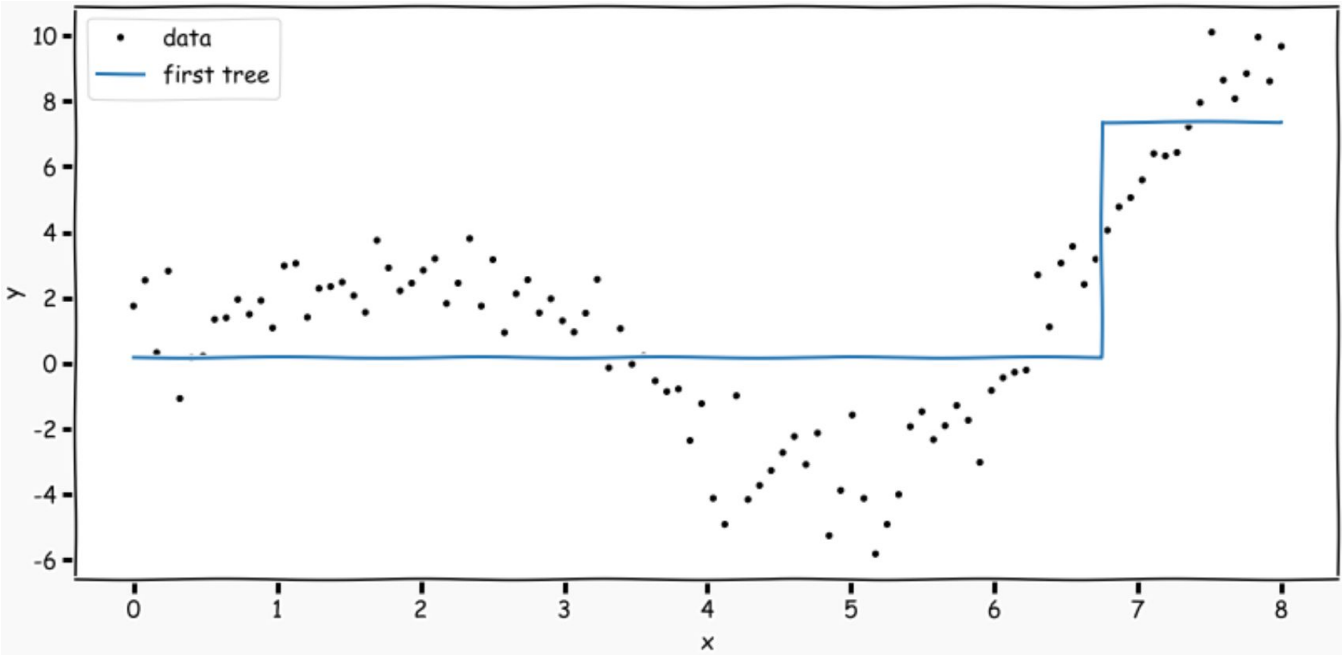
- The split with **lower total loss is selected** → equivalent to select the split that **minimizes the variance**

- The process is iterated by dividing the data in smaller and smaller rectangles until the stopping criterion is satisfied

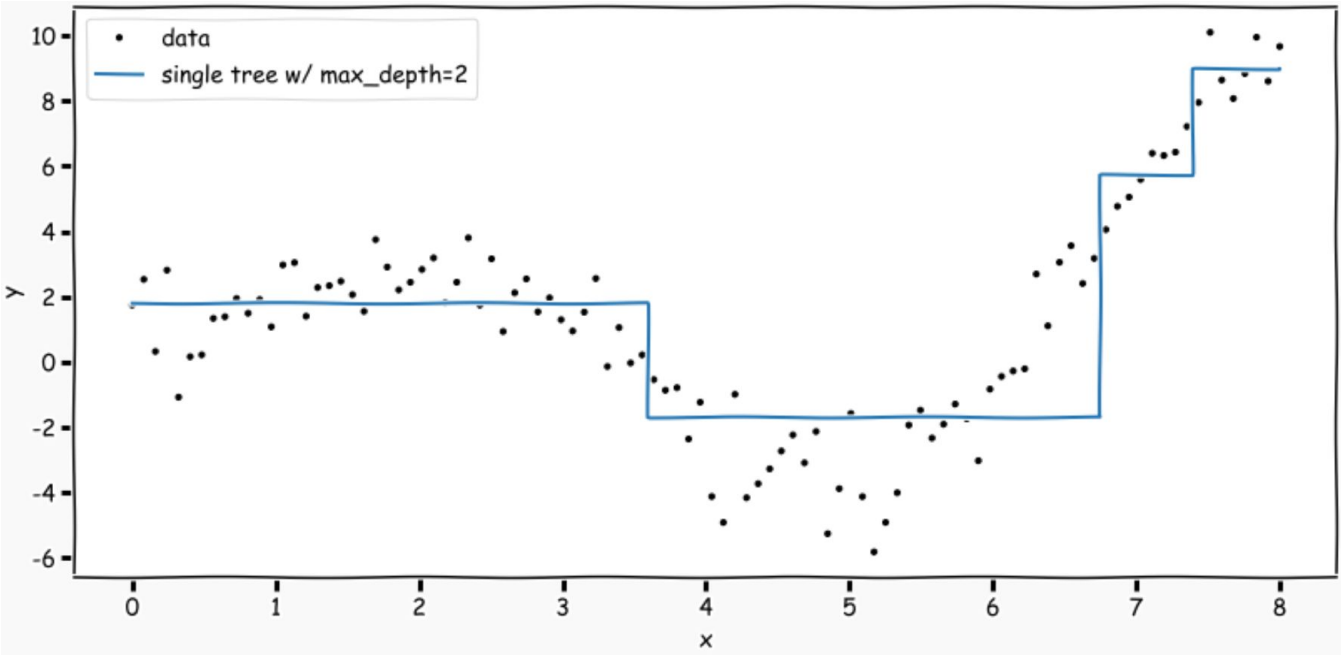
Regression trees **prediction**

- For any data point x_i
  - Traverse the tree until we reach a leaf node
  - Averaged value of the response variable y's in the leaf (from training set) is the prediction

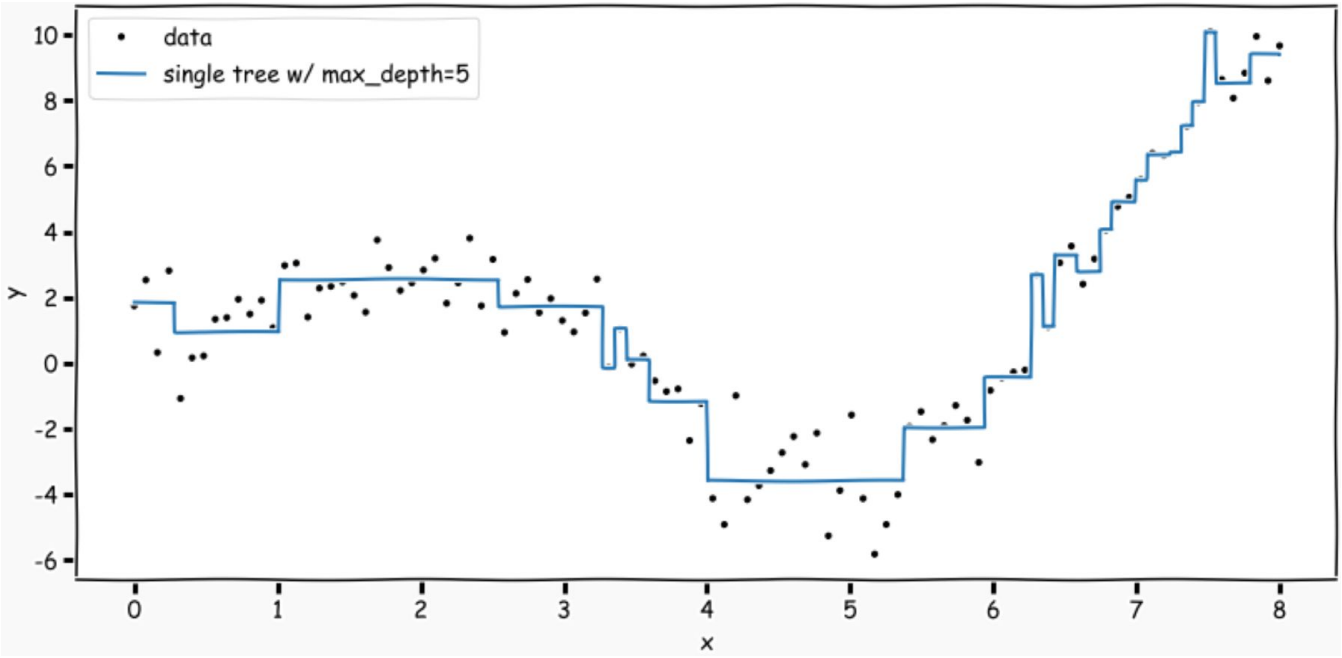
Single tree, max_depth == 1



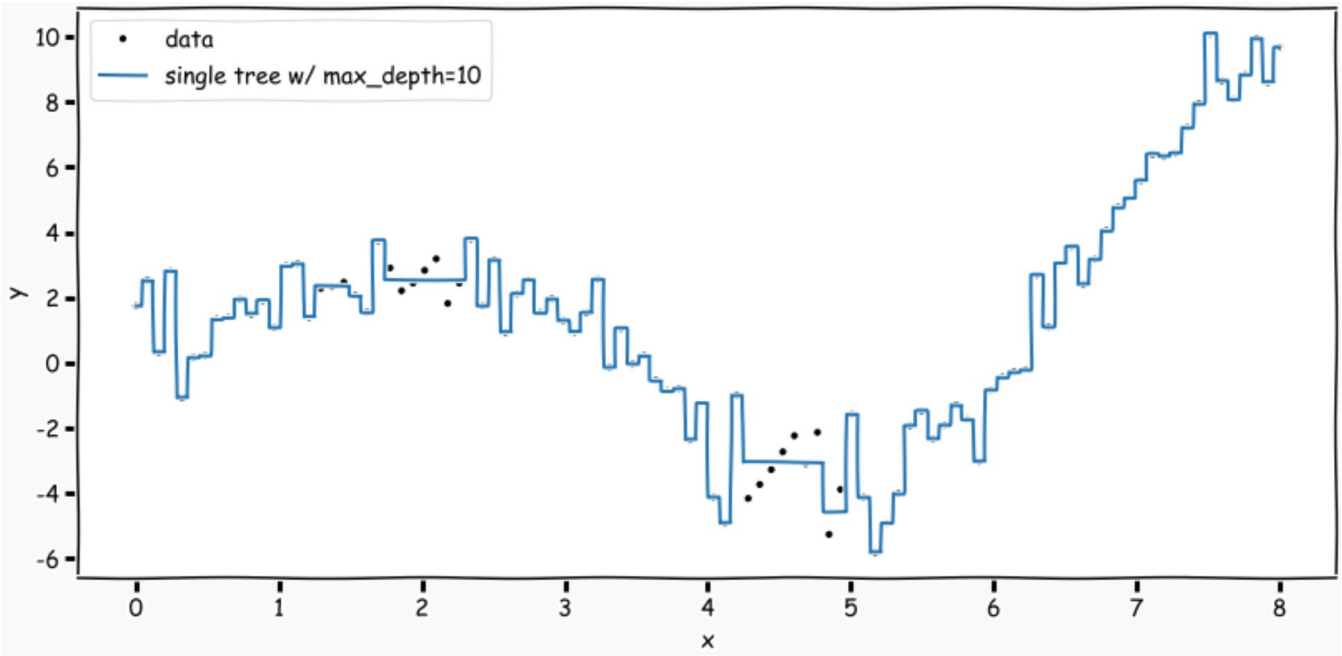
Single tree, max_depth == 2



Single tree, max_depth == 5



Single tree, max_depth == 10



Single decision trees

- **Pros**:
  - Requires little data preparation (unlike neural networks)
  - Can use continuous and categorical inputs

- **Cons**:
  - Danger of overfitting training data
  - Sensitive to fluctuations in training data
  - Hard to find global optimum
  - When to stop slitting?

Ensemble methods: combine **weak learners**

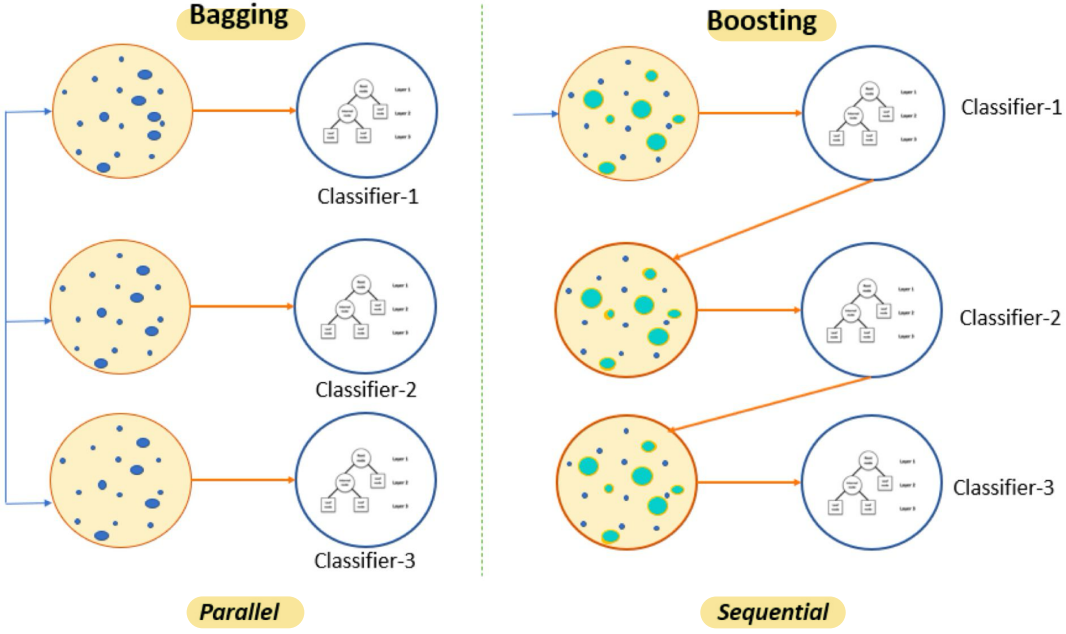
- **Bootstrap aggregation (bagging)**
  - Sample training data (with replacement) and train a (minimally regularized) tree on each of the derived training sets ⇒ high variance, low bias
  - Classify example with majority vote or compute average output from each tree as model output
  - **Reduce variance of low-bias models**

$$f(x) = \frac{1}{N_{\text{trees}}} \sum_{i=1}^{N_{\text{trees}}} f_i(x)$$

- **Boosting**
  - Train (highly regularized) models in sequence, giving more weight to examples not correctly classified by previous model ⇒ high bias, low variance
  - Take weighted average to classify examples
  - **Reduce bias of low-variance models**

$$f(x) = \sum_{i=1}^{N_{\text{trees}}} \alpha_i f_i(x)$$

Bagging vs Boosting



Tree boosting

- Each tree is created iteratively
- Tree's output $f_t(x)$ is given a **weight w_i relative to its accuracy**
- The ensemble output is the weighted sum

$$f(x) = \sum_{t=1}^{N_{\text{trees}}} \alpha_t f_t(x)$$

- After each iteration, each data sample is **given a weight based on how often it is missclassified**
- Goal is to minimize the objective function

$$l(S) = \sum_{i=1}^{N} L(f(x_i), x_i) + \sum_{t=1}^{N_{\text{trees}}} \Omega(f_t)$$

Loss function

Regularization term

Types of boosting - some pointers

- Adaptive boosting (AdaBoost)
  - One of the originals
  - Freund and Schapire (1997): 10.1006/jcss.1997.1504

- Gradient boosting
  - Uses gradient descent to create new learners
  - The loss function is differentiable
  - Friedman (2001): 10.1214/aos/1013203451

- Extreme gradient boosting (**XGBoost**, state of the art)
  - Very popular in data science (Kaggle) competitions
  - Chen and Guestrin (2016): 10.1145/2939672.2939785

Tunable parameters

- **Loss function:** How to define the distance between the truth and the prediction
  - Use binary logistic when you have two classes
- **Learning rate:** how much to adjust data weights after each iteration
  - Smaller is better but slower
- **Subsample size:** How many samples to train each new tree
  - Data samples are randomly selected each iteration
- **Number of trees:** How many total trees to create
  - This is the same as the number of iterations
  - Usually more is better, but could lead to overfitting

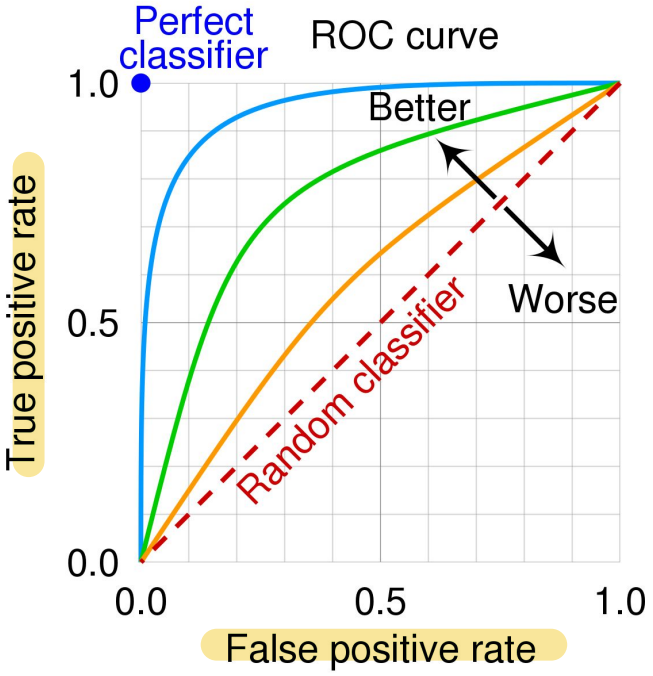
Parenthesis: the **ROC curve** for binary classifiers

- A ROC curve plots the **performance of a binary classification model at all classification thresholds**. It shows the trade-off between sensitivity and specificity

```
TPR = TP / (TP + FN) = Sensitivity

FPR = FP / (FP + TN) = 1 - Specificity
```

- **AUC (Area Under Curve)** summarizes the entire ROC curve into a single number between 0 and 1.



In conclusion: some **advantages of decision trees**

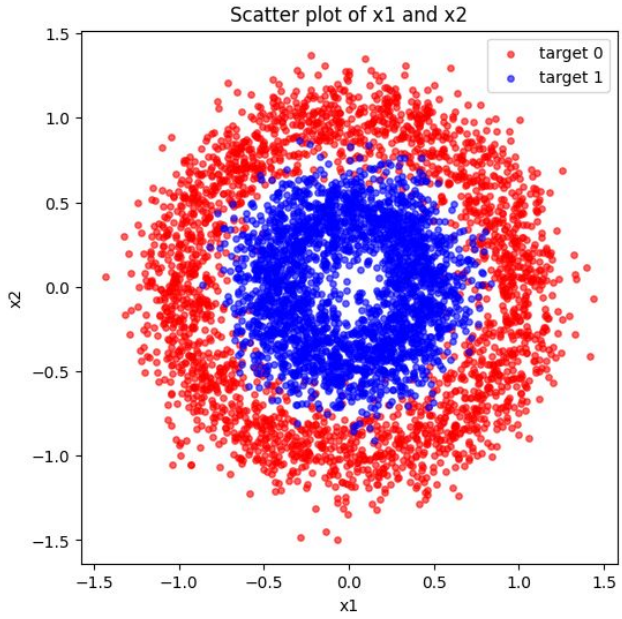
- Very easy to interpret!
- Closer to human decision-making
- Easy to visualize graphically
- They easily handle qualitative predictors and missing data

Example…

Circular correlation with trees

- Can I separate the two classes with a linear model?
  - Output is a **linear combination of the parameters**

- Let's use this notebook and implement a tree to perform classification



- Available classes
  - Single tree
    - `DecisionTreeClassifier` → uses gini by default
    - `DecisionTreeRegressor` → uses MSE by default
  - Ensembles
    - `AdaBoostClassifier` → makes use of the `DecisionTreeClassifier`
    - `AdaBoostRegressor` → makes use of the `DecisionTreeRegressor`

Hands on…

Classify signal vs bkg events using a BDT

- Same higgs dataset for classification
- **Build and train a tree based algo** and use it to separate signal vs bkg
- Highlight the feature importance from BDT

- Run lasso on the same dataset (previous exercise)
- Show the **correlation between lasso coefficients and feature importance**
  - Do they agree?
  - Why higher level variables seem to be more effective in discriminating s/b?

- Which approach does better in terms of accuracy?
  - N.B.: for ridge and lasso one should consider the logistic regression with a penalty term
  - logistic regression is basically a BCE with a sigmoid function at the end

Separate higgs events from background events

Reminder

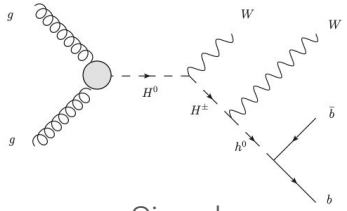
- A dataset is available where **collision events are generated** using a MC simulation

- Signal and bkg events have the **same final state** → separation is possible based on the **kinematic features of the particles** in the final state
  - 2 W bosons and 2 b quarks

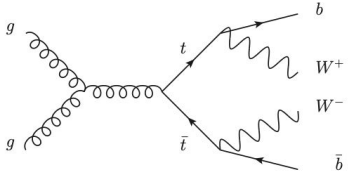
- **Goal:** distinguish exotic Higgs events from bkg involving top quarks

- The dataset can be accessed directly with the following syntax:

```
from sklearn.datasets import fetch_openml
# Load the Higgs dataset
higgs_data = fetch_openml(name='higgs', version=1, as_frame=True)
df = higgs_data.frame
```



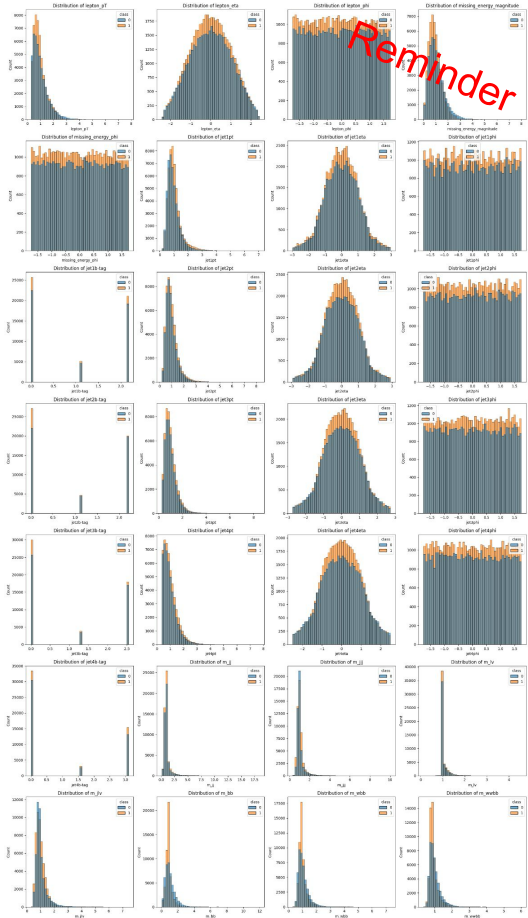
Signal



Background

Typical variables from an experiment at LHC

- More info on the dataset available
  - On the openml website
  - In the original paper from 2014

- Features include:
  - 21 low-level features (kinematic properties):
    - lepton pT, lepton eta, lepton phi, missing energy magnitude, missing energy phi, jet 1 pt, jet 1 eta, jet 1 phi, jet 1 b-tag, jet 2 pt, jet 2 eta, jet 2 phi, jet 2 b-tag, jet 3 pt, jet 3 eta, jet 3 phi, jet 3 b-tag, jet 4 pt, jet 4 eta, jet 4 phi, jet 4 b-tag
  - 7 high-level features derived by physicists:
    - m_jj, m_jjj, m_lv, m_jlv, m_bb, m_wbb, m_wwbb
  - The first column is the class label (1 for signal, 0 for background)

- To be noticed that the **features in the dataset have been scaled** wrt the original distributions
  - A lepton with a pt of 1GeV doesn't even reach the calorimeters